

Course project - Optimal GPU-CPU Offloading Strategies for Deep Neural Network Training

Olle Flygar

January 15, 2026

Contents

Contents	1
1 Introduction/Context	2
1.1 Scope of report	2
1.2 Deep neural networks	2
1.3 Training of deep neural networks	2
1.4 Problem introduction	2
2 State of the art	3
2.1 Checkpointing(Recomputation)	3
2.2 Offloading(swapping)	3
3 Overview of article contributions	4
3.1 Relaxation 1	5
3.2 Relaxation 2	5
3.3 Results	5
4 Detailed analysis of selected part - explanation of Algorithm 1	6
4.1 Why dynamic programming(DP)?	6
4.2 Algorithm explanation	6
5 Critical Evaluation and Discussion(Feedback)	7
5.1 Structure	7
5.2 Assumptions and model limitations	7
5.3 Strength of the approach	7
6 Bibliography	8

1 Introduction/Context

1.1 Scope of report

The primary aim of this report is to give an overview of the article by Beaumont et al.(1) and the GPU memory optimization problem in deep neural network training.

1.2 Deep neural networks

The core idea of deep neural networks(DNNs) consists of learning complex patterns and representations from vast amounts of data. DNNs has a large number of applications such as natural language processing, medical data analysis and image recognition. This has made DNNs widely popular and therefore also the use of them.

Architecturally the networks are composed of multiple layers consisting of nodes, usually referred to neurons or units in this context. Generally, neurons are connected to a subset of neurons in the preceding and/or following layers. For simplicity, the layers can be divided into three different types. One input layer receiving raw data which is passed into the network. One output layer producing the final prediction, for example a classification or a prediction. The input and output layers do not contain trainable parameters. Sandwiched between the input and output layer are two or more hidden layers. Neurons in these layers are essentially the ones responsible for learning patterns and representations by updating their weights and biases during training.

1.3 Training of deep neural networks

Training DNNs is known to be an expensive operation, with high memory usage and significant computational workload. The training involves two main phases, forward propagation and backwards propagation.

During forward propagation the input is passed through the network layer by layer and at each layer so called activations are computed. The activations serve as input to the next layer and are used to compute gradients during backwards propagation. At the end of the forward propagation phase the output is used to calculate a loss via a loss function. During the backward propagation phase layers are again processed one by one. For each layer corresponding gradients are computed using the stored activations and loss. Gradients are thereafter used to update the weights and biases of the neurons.

1.4 Problem introduction

There is one main reason why the training phase is so memory intensive: all activations computed during the the forward propagation phase must be stored in memory until they are needed in the corresponding backwards propagation step. In standard training setup, activations are stored on the GPU memory, as this is where the training is performed.

Deeper and larger networks, larger inputs(higher resolution images and/or videos) and larger batch sizes often provide several advantages, such as faster convergence, reduced training time, and more stable gradients leading to improved accuracy and more efficient training. This creates a problem because deeper networks, higher resolution images and larger training batch sizes are all factors that increase the number of activations needed to be stored on the GPU memory.

As a result, despite massive computational throughput of modern hardware, GPU memory acts as the primary bottleneck(during training of large and deep networks) rather than the computational workload. Data scientists are therefore prevented from considering the high performance networks they desire. This since the resulting increase in activations would exceed the current GPU hardware limits.

Given this it becomes clear that optimizing the GPU memory utilization would make it possible to consider the high performance networks desired. Solving or optimizing the GPU memory problem would therefore be of great value which is why this paper and several others aim to solve this problem. To clarify further, optimizing the utilization of the GPU memory is the problem the authors of this article try to solve.

Several techniques and approaches have been considered prior to the one considered in this article. Some works propose memory saving techniques or layer optimization. The two most popular approaches are offloading(swapping) where some activations are transferred from GPU to CPU, and checkpointing(recomputation) where only some activations are saved and the rest are recomputed during backwards propagation.

2 State of the art

In 2020, when the considered paper was written, the state of the art in optimal GPU memory utilization during DNN training primarily focused on training on a single GPU/node. The proposed techniques at the time mainly considered two approaches, checkpointing(recomputation) and offloading/swapping.

These two approaches, as well as previous ones, have the drawback of introducing some overhead cost, either recomputation(checkpointing) or communication delays(offloading). They are nevertheless interesting as they are suitable for training any neural network with limited resources, only one GPU/node.

Since the considered article focuses on the offloading approach the SOTA of the offloading approach is what is mainly considered.

2.1 Checkpointing(Recomputation)

We first consider the checkpointing(recomputation) approach. This technique consists of storing only a few of the activations computed in the forward propagation phase, reducing the memory needed. Then, during backward propagation, absent values are recomputed. This technique introduces a tradeoff between memory usage and computational cost.

2.2 Offloading(swapping)

Offloading(swapping) is the second popular approach to reduce the GPU memory requirements. This is accomplished by transferring some activations from the GPU to the CPU in the forward propagation phase and prefetching them during the backwards propagation phase. The challenge lies in deciding which activations to offload and when. Previous approaches leading up to this work propose heuristic solutions to the problem, generally not proving an optimal offloading schedule, resulting in unnecessary overhead.

The idea of offloading was first introduced by vDNN[17] developed by people at Nvidia. This approach either offload all activations or only activations corresponding to convo-

lutional layers. Here offloading is performed synchronously (still as soon as possible), which introduced some unnecessary idle times. This approach still showed improvements, reducing the average GPU memory usage with 90% with only an 18% performance loss (compared a hypothetical GPU able to hold the entire memory of the DNN).

vDNN was then further developed into vDNN++ [18] where several improvements were made, resolving identified issues from previous approach. In vDNN++ offloading is performed asynchronously. Decreasing GPU idle time even more. Furthermore the new approach uses compression on the CPU in order to decrease the needed memory. Lastly, memory fragmentation issues are further looked into by empirical evaluation. This improved algorithm, resulted in a 60% relative improvement from the previous vDNN approach. Although improvements were made the primary decision logic for offloading remains heuristic.

Two other considerable offloading approaches at the time were TFLMS [38] and AutoSwap [39]. Both of these provide some significant performance but are mostly based on heuristic strategies too. Both of them use swap in swap out operations. TFLMS is developed as a module in TensorFlow where offloading is based on the distance between nodes. TFLMS achieved some substantial results, being *able to train ResNet-50 and 3DUnet with with 4.7x and 2x larger batch size, respectively*.

AutoSwap is a CUDA based implementation, here the offloading is based on priority scores. Here Bayesian optimization is used to optimize these priority scores. However, since Bayesian optimization is a random procedure, the performance is very inconsistent. Some of the strategies in the AutoSwap paper still achieves considerable results, reducing the *memory cost by up to 34.2% without communication overhead*.

In summary, the state of the art at the time, had introduced some interesting offloading techniques decreasing the GPU memory requirements. Despite performing quite well, the primary decision logic is heuristic.

3 Overview of article contributions

To understand the overall contributions it is necessary to understand how the approach is structured. To derive an optimal solution of the offloading problem the authors first of all strictly defines it. The problem is defined as a sequential decision problem with memory and communication constraints.

After the problem formulation, the authors go on to prove that the problem is strongly NP-hard by a reduction from the 3-partition problem. **The reduced offloading problem is constructed such that a solution exists if and only if the 3-partition problem has a solution.** Since the 3-partition problem is strongly NP-hard, it follows that the offloading problem also is strongly NP-hard.

Proving NP-hardness in the strong sense implicates that an optimal algorithm generally is infeasible, unless $P = NP$. Because of this, problem relaxations are justified.

Theoretically grounding the problem by proving the complexity of it constitutes one of the articles most significant contributions. Contrary to previous work (vDNN, vDNN++, TFLMS and AutoSwap) where mainly heuristic strategies are considered without accompanying theoretical justification.

3.1 Relaxation 1

Based on the proven problem complexity, two relaxations of the problem are introduced, the first one which leads to the Greedy approach. This relaxation allows for activations to be partially offloaded, enabling the problem to be formulated in a continuous form. An eager and ordered schedule is proven to be the optimal structure of the problem, implying that activations should be offloaded as soon as possible and in increasing order. Based on this structure an optimal algorithm is derived thru formal proofs. The optimal heuristic algorithm Greedy returns a no-wait, eager, ordered schedule where the first k activations are offloaded entirely until memory constraints are sure to be satisfied, i.e offloading at least $(Memory_{peak} - Memory_{GPU})$ to the CPU.

3.2 Relaxation 2

For a more precise algorithm, a second, more sophisticated problem relaxation is considered. Here, activations are required to be entirely offloaded, but are allowed to be paused and resumed later. Activations can then be scheduled over time, which results in a more realistic model. The best ordered schedule is found thru deriving properties of all *ordered and no wait schedules*. From this the authors are able to obtain an optimal DP algorithm, called DynProg, resulting in algorithm 1. In contrast to previous approaches, even this relaxed approach relies heavily on theoretical deduction, rather than heuristic strategies.

3.3 Results

To evaluate the derived relaxed approaches, an extensive experimental evaluation was conducted on the respective implemented algorithms. Here, the two derived heuristic algorithms, Greedy and DynProg, were compared against state of the art at the time, vDNN, TFLMS and AutoSwap. Results from the stated algorithms were obtained on three different networks(ResNet, DenseNet and Inception) with varying settings(image sizes and batch size).

Despite the low complexity of the Greedy algorithm it still shows strong results, often performing similar to AutoSwap while having significantly lower computational cost.

Among the five evaluated approaches the theoretically grounded DynProg achieves the overall best performance, being considerable more stable than vDNN and AutoSwap, and performing better than TFLMS across a wider range. TFLMS fails to adapt to low memory settings with limited bandwidth. Dynprog also significantly outperforms the Greedy algorithm. Among the five approaches, vDNN performs the poorest, outperformed by both Greedy and DynProg in all cases, especially low memory ones. Notably, DynProg also achieves a consistent throughput of at least 83%.

Given this, the results clearly show that both Greedy and DynProg provide significant improvements compared to previous approaches.

4 Detailed analysis of selected part - explanation of Algorithm 1

Perhaps not completely unexpected, the primary aim of algorithm 1 is to solve the main problem of determining which activations to offload from the GPU to the CPU and which to not. As described above, algorithm 1 is based on the second, more sophisticated, relaxation of the problem where it is defined as an optimal scheduling problem. The relaxation assumes that the schedule is ordered and under a no-wait policy, i.e transfers are performed in order and as early as possible.

For simplicity, algorithm 1 can be described as a basis for selecting the optimal offloading schedule, by minimizing GPU idle time thru dynamic programming(DP).

4.1 Why dynamic programming(DP)?

The authors clearly defines the core problem as a scheduling problem, or a sequential decision problem, where in each layer a binary decision either to offload or not is taken. From this the authors show that the offloading decision in each layer can be solely based on the current GPU memory occupancy, and the amount of data awaited from offloading and prefetching. Defining each layers binary decision solely based on current and future values is well suited for dynamic programming when defining each layer with corresponding binary decision as a state.

For additional clarification, it may be useful to briefly recall why neither a greedy nor an exhaustive solution is realistic. A greedy algorithm would consider the benefits of immediate offloading without considering future computational costs. At the same time, exhaustively solving all possible offloading patterns would be computationally insoluble. Thus, neither this nor the greedy algorithm is a feasible solution. For this reason, a DP algorithm once again becomes the natural choice.

4.2 Algorithm explanation

In algorithm 1 the binary offloading decision is based on the state(markov property) of layer i and is defined as $(M_{F_i}, \Delta_{F_i}, \Delta_{B_i})$ where

- M_{F_i} = Current GPU memory occupancy
- Δ_{F_i} = pending data from offloading
- Δ_{B_i} = pending data from prefetching

The state can be interpreted as the total minimum accumulated GPU idle time up to that state, which is what we want to minimize. Since the total computation time of the backwards and forward propagation phase is fixed, minimizing the GPU idle time is equivalent to minimizing the total execution time, which is why this is the objective. Furthermore the algorithm is iterated over all i states(layers).

To update state i , algorithm 1 first of all checks whether the current state is feasible or not, i.e can following forward and backwards operations be executed without exceeding the GPU memory limits. Checking this is necessary to ensure layer i can even be executed at all.

The algorithm then explores how offloading or not in state i affects the total accumulated

GPU idle time in the subsequent state $i+1$. From equations (1) and (6), the idle times ϵ_i^F and ϵ_i^B , during the transition between state i and $i+1$, are computed for the corresponding step in the forward and backward propagation phases. ϵ_i^F and ϵ_i^B are used to compute the total accumulated idle times for the two possible states at layer $i+1$, where offloading is performed or not. The next state variables are computed from equations (2), (3) and (7) for offloading and equation (4), (5) and (8) otherwise. The two state representations are stored in hash table $i+1$.

Furthermore, after iterating thru all layers the idle time ϵ_G between the forward and backward propagation phase is computed resulting in a total idle time.

Finally the hash table is backtracked to retain the optimal offloading decision schedule.

5 Critical Evaluation and Discussion(Feedback)

5.1 Structure

The structure of the report is overall interpretable and is possible to understand. As in many academic articles it takes time to capture the essence of the article as well as the theory behind it.

Something that makes the theory especially hard to understand is that all variables are defined one time and one time only before they are used in the rest of the article. Keeping track of what each variable represents throughout the article becomes hard since there are so many. This makes proofs and derivations challenging to follow.

5.2 Assumptions and model limitations

There are several assumptions and limitations of the articles proposed approach.

The training is assumed to be performed on one GPU/node and since most training today is performed on large clusters with several GPUs this approach may be a limitations for some tasks. Furthermore the model assumes the deep neural network can be decomposed as a sequential graph. This is not be applicable to some present complex DNN architectures. Lastly the article assumes perfect overlap between communication and computation which for the most part is realistic in practice. However, certain situations could cause overhead this article does not account for, for example overhead in software frameworks or smaller kernels.

5.3 Strength of the approach

The considered paper is the first one to base an algorithm on theoretical deduction, rather than heuristic strategies. I think that proving a near optimal algorithm thru theoretical deduction is what makes this article stand out against state of the art at the time. Other than strong theoretical deduction the DynProg and Greedy algorithm show some substantial performance against state of the art models, DynProg showing the overall best performance out of the five models tested.

6 Bibliography

- [1] O. Beaumont, L. Eyraud-Dubois, and A. Shilova, “Optimal gpu-cpu offloading strategies for deep neural network training,” Feb. 2020, research paper.